JobSheet - Week 2 - Object, Class & Encapsulation

Nama: Muhammad Fawwaz Muzakki

NIM: 251524056

Kelas: 1B

Repo GitHub:
https://github.com/JackAprocat/TekProm_Praktek/tree/main/Week%202

Instruksi Pengerjaan:

1. Kerjakan 5 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W2_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W2_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Evaluasi & Refactoring Class Restaurant

Soal Praktikum

Diketahui terdapat dua file program yang digunakan untuk menyimpan daftar menu makanan dan stoknya, yaitu Restaurant.java & RestaurantMain.java.
 	
1. Restaurant.java
public class Restaurant {
public String[] nama_makanan;
public double[] harga_makanan;
public int[] stok;
public static byte id = 0;
 	
public Restaurant() {
nama_makanan = new String[10];
harga_makanan = new double[10];
stok = new int[10];

```
    }
}

public void tambahMenuMakanan(String nama, double harga, int stok) {
    this.nama_makanan[id] = nama;
    this.harga_makanan[id] = harga;
    this.stok[id] = stok;
}

public void tampilMenuMakanan() {
    for (int i = 0; i <= id; i++) {
        if (!isOutOfStock(i)) {
            System.out.println(
                nama_makanan[i] + "[" + stok[i] + "]" + "\tRp. " + harga_makanan[i]
            );
        }
    }
}

public boolean isOutOfStock(int id) {
    if (stok[id] == 0) {
        return true;
    } else {
        return false;
    }
}

public static void nextId() {
    id++;
}
```

2. RestaurantMain.java

```
public class RestaurantMain {
    public static void main(String[] args) {
        Restaurant menu = new Restaurant();

        menu.tambahMenuMakanan("Pizza", 250000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Spaghetti", 80000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Tenderloin Steak", 60000, 30);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Chicken Steak", 45000, 30);

        menu.tampilMenuMakanan();
    }
}
```

A. Instruksi - Analisis Desain Class:

Amati kode program yang diberikan, kemudian jawab pertanyaan berikut:

1. Apakah class Restaurant sudah menerapkan Encapsulation dengan benar? Anda dapat mengacu pada Buku Ajar Tekpro (Teori) - Chapter 2.4 Enkapsulasi

(Lihat di Buku Acuan pada Google Classroom) atau Buku Java Core Design Hint Chapter 4.10.

2. Apakah attribute yang diterapkan saat ini aman dari akses secara langsung?
3. Apakah terdapat pelanggaran prinsip OOP (misalnya public attribute)?

B. Instruksi - Perbaikan Kode Program:

Lakukan perbaikan desain class dengan ketentuan:

1. Semua attribute harus bersifat private
2. Akses data dilakukan melalui getter dan setter
3. Validasi stok (stok tidak boleh negatif)
4. Pengembangan Fitur (Mini Case) Tambahkan fitur berikut:
   a) Pemesanan menu
   b) Stok otomatis berkurang setelah pemesanan
   c) Pesan ditolak jika stok tidak mencukupi

Source Code

1. Restaurant.java

```
public class Restaurant {
  private String[] nama_makanan;
  private double[] harga_makanan;
  private int[] stok;
  private static byte id = 0;

  public Restaurant() {
    nama_makanan = new String[10];
    harga_makanan = new double[10];
    stok = new int[10];
  }

  public void tambahMenuMakanan(String nama, double harga, int stok) {
  this.nama_makanan[id] = nama;
  this.harga_makanan[id] = harga;
  this.stok[id] = stok;
  }

  public void pesananMenu(int id, double harga, int jumlah) {

    if (jumlah > 0){
      int stokAfterBuy = this.stok[id] - jumlah;

      if( stokAfterBuy >= 0){
        this.stok[id] = stokAfterBuy;
        System.out.println(this.nama_makanan[id] + "[" + this.stok[id] + "]");
        double totalHarga = harga * jumlah;
        System.out.println("Total Harga: " + totalHarga);
        System.out.println();
      } else {
```

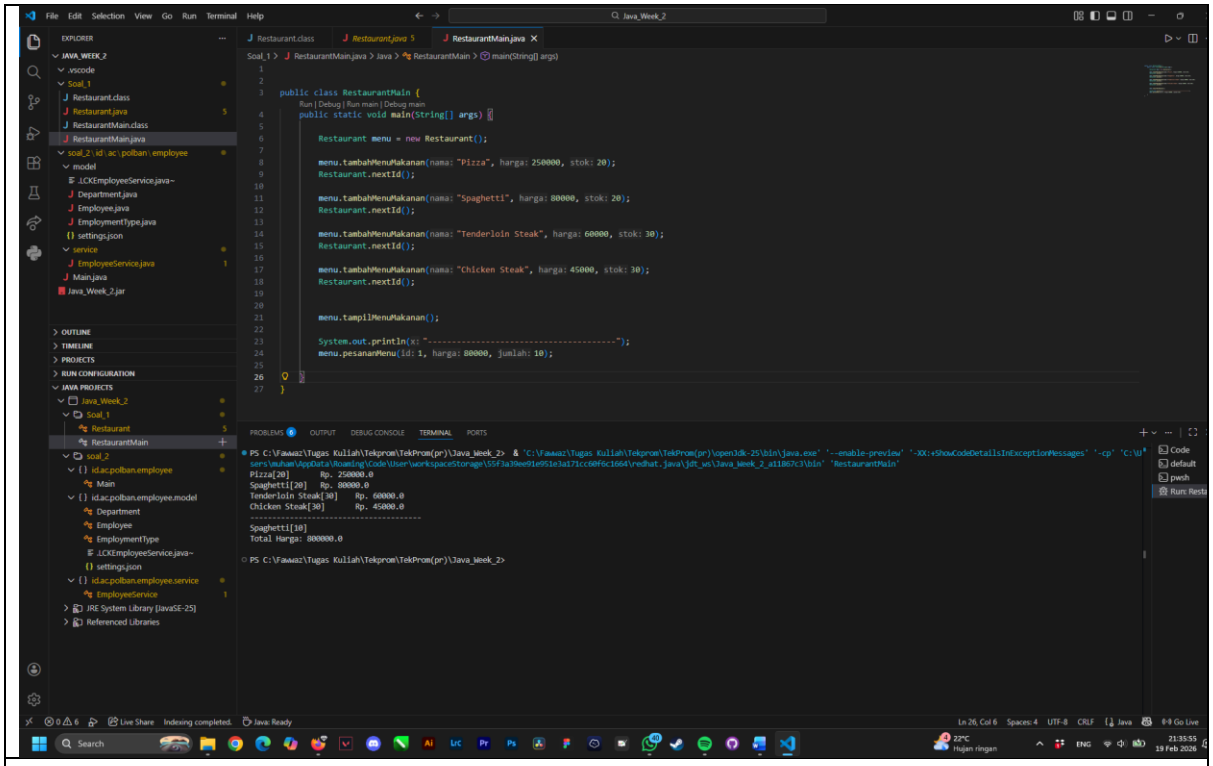
```
            System.out.println("Stok tidak memenuhi pesanan");
        }

    } else {
        System.out.println("Jumlah tidak valid");
    }
}

public boolean validasiStok(int id){
    if (stok[id] <= 0){
        return true;
    } else {
        return false;
    }
}

public String getnNamaMakananById(int id){
    return nama_makanan[id];
}

public double getHargaMakanan(int id){
    return harga_makanan[id];
}

public void tampilkanHarga(){
    double hargaMakanan = getHargaMakanan( 1);
    System.out.println("Harganya adalah: " + hargaMakanan);
}

public void setStockById(int id, int stok){
    if(stok >= 0){
        this.stok[id] = stok;
    }
}
public void tampilMenuMakanan() {
    for (int i = 0; i <= id; i++) {
        if (!isOutOfStock(i)) {
            System.out.println(
                nama_makanan[i] + "[" + stok[i] + "]" + "\tRp. " + harga_makanan[i]
            );
        }
    }
}



public boolean isOutOfStock(int id) {
```

```
        if (stok[id] == 0) {
            return true;
        } else {
            return false;
        }
    }

    public static void nextId() {
        id++;
    }

}
```

2. RestaurantMain.java

```
public class RestaurantMain {
    public static void main(String[] args) {

        Restaurant menu = new Restaurant();

        menu.tambahMenuMakanan("Pizza", 250000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Spaghetti", 80000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Tenderloin Steak", 60000, 30);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Chicken Steak", 45000, 30);
        Restaurant.nextId();

        menu.tampilMenuMakanan();

        System.out.println("----------------------------------------");
        menu.pesananMenu(1, 80000, 10);

    }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

A.

1. Code di atas belum menerapkan enkapsulasi dengan benar karena attribut menggunakan public semua seharusnya menggunakan private agar tidak bisa di buah dari luar method
2. Tidak aman karena menggunakan public
3. Ada, public attribute dan public static atribut

B.

1. Permasalahan: Pada kode awal, beberapa atribut seperti id dan array data bisa diakses secara tidak terkontrol. Hal ini berisiko membuat data tidak konsisten.

   Solusi: Mengubah semua akses atribut (seperti nama_makanan, harga_makanan, stok) menjadi private. Dengan begini, data hanya bisa diakses atau diubah melalui metode resmi yang disediakan oleh kelas tersebut.

2. Permasalahan: Sebelumnya, Anda mengisi data dengan langsung menunjuk indeks array berdasarkan variabel id yang diatur manual di luar kelas. Ini menyulitkan jika kita ingin mengambil informasi spesifik secara aman.

   Solusi: Menyediakan metode getNama(), getHarga(), dan setStok(). Penggunaan getter memudahkan pengambilan data tanpa risiko mengubahnya, sedangkan setter memberikan gerbang kendali saat data hendak diubah.

3. Permasalahan: Pada kode awal, jika ada input stok -5, sistem tetap akan menerimanya. Logika stokAfterBuy juga hanya mengecek saat pembelian,

namun tidak mencegah jika ada orang yang sengaja mengubah stok menjadi negatif melalui metode setStockById.

Solusi: Menambahkan Logika Validasi di dalam metode setStok. Sebelum nilai stok baru disimpan ke dalam array, program akan mengecek: if (stok >= 0). Jika nilainya di bawah 0, perubahan akan ditolak atau diberikan peringatan, sehingga integritas data terjaga.

4. Permasalahan: Metode pesananMenu Anda sebelumnya mengharuskan pengguna memasukkan harga secara manual di parameter (int id, double harga, int jumlah). Ini tidak efisien karena harga seharusnya sudah tersimpan di sistem. Selain itu, pengurangan stok dilakukan secara manual di dalam metode utama.

   Solusi:
   - Penyederhanaan Parameter: Metode pesanan kini hanya butuh (id, jumlah). Sistem akan mencari harga secara mandiri berdasarkan ID yang diberikan.
   - Otomasi: Begitu jumlah pesanan valid dan stok cukup, sistem langsung menjalankan perintah this.stok[id] -= jumlah.
   - Pesan Penolakan: Jika hasil pengurangan menghasilkan nilai di bawah nol, sistem akan mencetak pesan: "Pesanan ditolak karena stok tidak mencukupi", dan transaksi dibatalkan (stok tidak berkurang).

Nama Teman Hal yang Dibantu (Opsional)
Hafiz dan Iqbal (menjelaskan beberapa pertanyaan yang saya ajukan)

No. 2 Interaksi Object dan Struktur Class dalam Java

Soal Praktikum

Pada pertemuan sebelumnya, mahasiswa telah mempelajari konsep dasar dalam Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) yang menjadi fondasi utama dalam pengembangan aplikasi berbasis Java.

1. *Object*
   Object merupakan representasi nyata dari suatu entitas yang memiliki data (attribute) dan perilaku (method). Object dibuat dari sebuah class dan digunakan untuk menjalankan proses dalam program.
2. *Class*
   Class adalah cetak biru (blueprint) untuk membuat object. Class mendefinisikan struktur data dan perilaku yang dimiliki oleh object, sehingga memungkinkan pembuatan object dengan karakteristik yang sama.
3. *Message*
   Message menggambarkan proses komunikasi antar object. Dalam Java, message diwujudkan dalam bentuk pemanggilan method pada suatu object untuk meminta object tersebut melakukan suatu aksi.

4. *Encapsulation*
   Encapsulation adalah prinsip OOP yang digunakan untuk melindungi data dengan membatasi akses langsung terhadap attribute. Penerapan encapsulation dilakukan menggunakan access modifier dan accessor method (getter dan setter).

Pada pertemuan kali ini, mahasiswa akan mempelajari praktik beberapa konsep lanjutan dalam Pemrograman Berorientasi Objek menggunakan Java yang berfokus pada interaksi antar object serta struktur dan organisasi program.

1. *Relationship Between Class*
   Membahas hubungan antar class dalam sebuah sistem, seperti bagaimana satu class dapat menggunakan atau memiliki object dari class lain untuk membentuk sistem yang lebih kompleks.
2. *Static Field & Method*
   Mempelajari penggunaan field dan method yang bersifat static, yaitu milik class dan bukan milik object, serta memahami kapan dan mengapa konsep static digunakan.
3. *Method Parameters*
   Membahas cara pengiriman data ke dalam method melalui parameter, termasuk penggunaan object sebagai parameter untuk memungkinkan interaksi antar object.
4. *Object Construction*
   Mempelajari proses pembuatan object melalui constructor, termasuk inisialisasi attribute dan pengaturan kondisi awal object saat pertama kali dibuat.
5. *Packages*
   Membahas pengelompokan class ke dalam package untuk mengatur struktur program, meningkatkan keterbacaan kode, dan menghindari konflik nama class.
6. JAR
   Mempelajari konsep Java Archive (JAR) sebagai media untuk mengemas dan mendistribusikan aplikasi Java agar dapat dijalankan atau dibagikan dengan lebih mudah.

Ketentuan Praktikum:
Pada praktikum ini, mahasiswa diminta untuk memahami contoh program employee dan mengembangkan aplikasi tersebut dengan menerapkan beberapa konsep lanjutan dalam Pemrograman Berorientasi Objek menggunakan Java.

A. Instruksi - Tulis Ulang Kode Program Employee

1. Kelas Department

```
package id.ac.polban.employee.model;

public class Department {
 private String name;

 public Department(String name) {
    this.name = name;
```

```
 }

 public String getName() {
    return name;
 }

 public void setName(String name) {
    this.name = name;
 }
}
```

2. Kelas Employee

```
package id.ac.polban.employee.model;

public class Employee {
    private int id;
    private String name;
    private Department department;
    private EmploymentType type;
    private double salary;

    public Employee(int id, String name, Department department,
EmploymentType type, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.type = type;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Department getDepartment() {
        return department;
    }
    public void setDepartment(Department department) {
        this.department = department;
    }
    public EmploymentType getType() {
        return type;
    }
    public void setType(EmploymentType type) {
        this.type = type;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
```

```
}
```

3. Kelas EmployeeType

```
package id.ac.polban.employee.model;

public class EmploymentType {
    private String type;

    public EmploymentType(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}
```

4. Kelas EmployeeService

```
package id.ac.polban.employee.service;

import java.util.HashMap;
import java.util.Map;

import id.ac.polban.employee.model.*;

// mengelola operasi yang berkaitan dengan data dan aturan bisnis
public class EmployeeService {
    private Map<Integer, Employee> employees = new HashMap<>();

    public void addEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public Employee getEmployee(int id) {
        return employees.get(id);
    }

    public void raiseSalary(int id, double percent) {
        Employee emp = employees.get(id);
        if (emp != null) {
            emp.setSalary(emp.getSalary() * (1 + percent/100));
        }
    }
}
```

B. Instruksi - Lengkapi Studi Kasus, Rancangan Class Diagram & Penjelasan:

Adapun ketentuan dan tugas praktikum adalah sebagai berikut:

1. Lengkapi studi kasus yang telah dibuat dengan menerapkan penggunaan *static field* dan *static method* secara tepat!
2. Terapkan konsep package dengan membuat minimal dua package, yaitu:
    - `id.ac.polban.employee.model`
    - `id.ac.polban.employee.service`

3. Buat diagram kelas (**class diagram**) yang menggambarkan struktur class, attribute, method, serta hubungan antar class dalam sistem.
4. Implementasikan relasi antar class pada program, yang mencakup:
    - *Dependency*
    - *Aggregation*
5. Jelaskan perbedaan dan fungsi masing-masing jenis relasi tersebut berdasarkan kasus yang dibuat!
6. Lakukan proses generate aplikasi ke dalam file JAR!
7. Buat sebuah project Java baru yang hanya memuat file JAR hasil generate, kemudian jalankan aplikasi dari project tersebut untuk memastikan file JAR dapat digunakan dengan benar!
8. Seluruh source code, diagram kelas, serta penjelasan implementasi didokumentasikan pada jobsheet ini! Pastikan source code di submit pada repository GitHub masing-masing!

Source Code

1. Main.java

```
package id.ac.polban.employee;

import id.ac.polban.employee.model.*;
import id.ac.polban.employee.service.*;

public class Main {
  public static void main(String[] args) {
    System.out.println("=== SISTEM MANAJEMEN KARYAWAN ===\n");


    Department deptIT = new Department("Information Technology");
    EmploymentType typeFullTime = new EmploymentType("Full-Time");

    Employee emp1 = new Employee("Fawwaz", deptIT, typeFullTime, 5000000);
    Employee emp2 = new Employee("Hasan", deptIT, typeFullTime, 4500000);


    EmployeeService service = new EmployeeService();
    service.addEmployee(emp1);
    service.addEmployee(emp2);


    System.out.println("--- Data Karyawan ---");
    System.out.println("Karyawan 1: " + service.getEmployee(emp1.getId()).getName() + " | Dept: " + service.getEmployee(emp1.getId()).getDepartment().getName());
    System.out.println("Karyawan 2: " + service.getEmployee(emp2.getId()).getName() + " | Dept: " + service.getEmployee(emp2.getId()).getDepartment().getName());
```

```
        System.out.println("\n--- Proses Kenaikan Gaji ---");
            System.out.println("Gaji awal " + emp1.getName() + ": Rp" +
emp1.getSalary());
        service.raiseSalary(emp1.getId(), 10);
            System.out.println("Gaji baru " + emp1.getName() + ": Rp" +
emp1.getSalary());


        System.out.println("\n--- Statistik Perusahaan ---");
                System.out.println("Total Karyawan Terdaftar: " +
Employee.getCountEmployees());
    }
}
```

2. Departement.java

```
package id.ac.polban.employee.model;

public class Department {
 private String name;

 public Department(String name) {
   this.name = name;
 }

 public String getName() {
   return name;
 }

 public void setName(String name) {
   this.name = name;
 }
}
```

3. Employee.java

```
package id.ac.polban.employee.model;

public class Employee {
   private int id;
   private String name;
   private Department department;
   private EmploymentType type;
   private double salary;

   private static int lastId = 0;
```

```
    private static int countEmployees = 0;


     public Employee(String name, Department department, EmploymentType
type, double salary) {
    this.id = ++lastId;
    this.name = name;
    this.department = department;
    this.type = type;
    this.salary = salary;

    countEmployees++;
}

    public static int getCountEmployees(){
      return countEmployees;
    }

    public int getId() {
      return id;
    }
    public void setId(int id) {
      this.id = id;
    }
    public String getName() {
      return name;
    }
    public void setName(String name) {
      this.name = name;
    }
    public Department getDepartment() {
      return department;
    }
    public void setDepartment(Department department) {
      this.department = department;
    }
    public EmploymentType getType() {
      return type;
    }
    public void setType(EmploymentType type) {
      this.type = type;
    }
    public double getSalary() {
      return salary;
    }
    public void setSalary(double salary) {
      this.salary = salary;
```

```
    }
}
```

4. EmploymentType.java

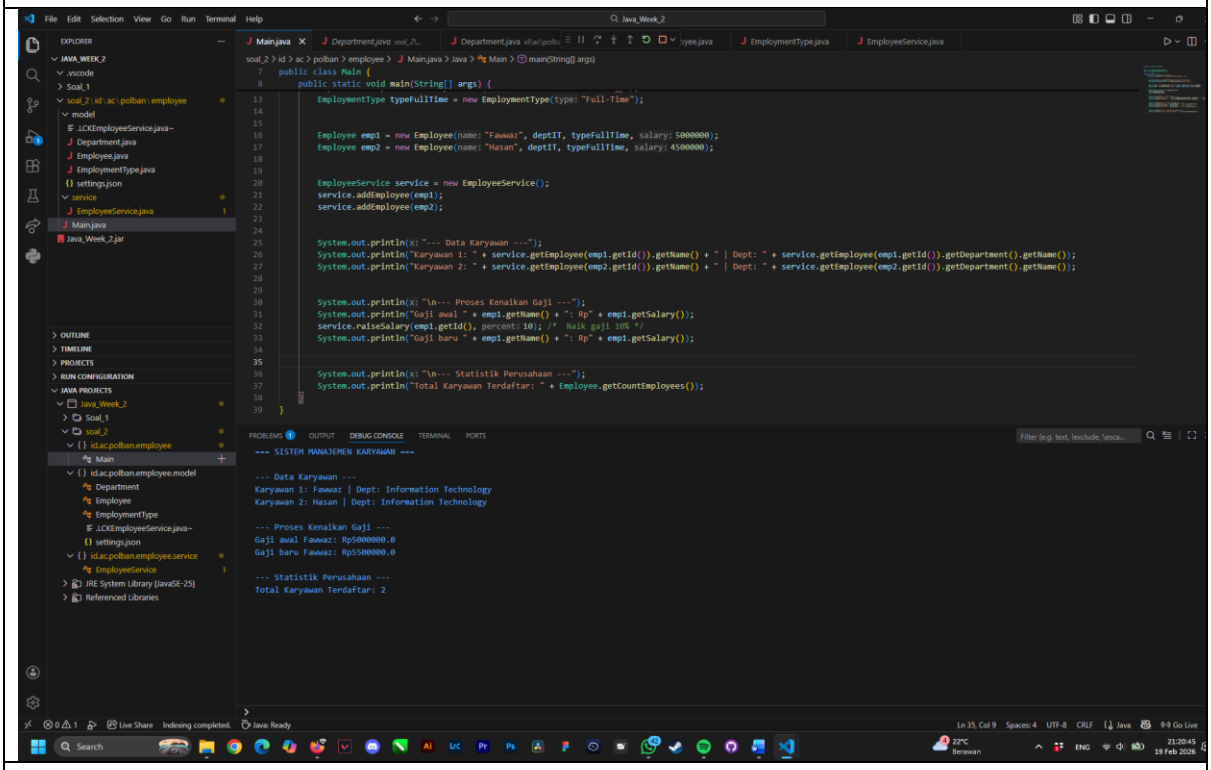
```
package id.ac.polban.employee.model;

public class EmploymentType {
    private String type;

    public EmploymentType(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}
```

5. EmployeeService.java

```
package id.ac.polban.employee.service;

import id.ac.polban.employee.model.*;
import java.util.HashMap;
import java.util.Map;

// mengelola operasi yang berkaitan dengan data dan aturan bisnis
public class EmployeeService {
    private Map<Integer, Employee> employees = new HashMap<>();

    public void addEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public Employee getEmployee(int id) {
        return employees.get(id);
    }

    public void raiseSalary(int id, double percent) {
        Employee emp = employees.get(id);
        if (emp != null) {
            emp.setSalary(emp.getSalary() * (1 + percent/100));
```

```
        }
    }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

1. Permasalahan: Bagaimana cara mengelola data global seperti jumlah total karyawan dan otomasi ID unik tanpa input manual setiap kali objek dibuat?

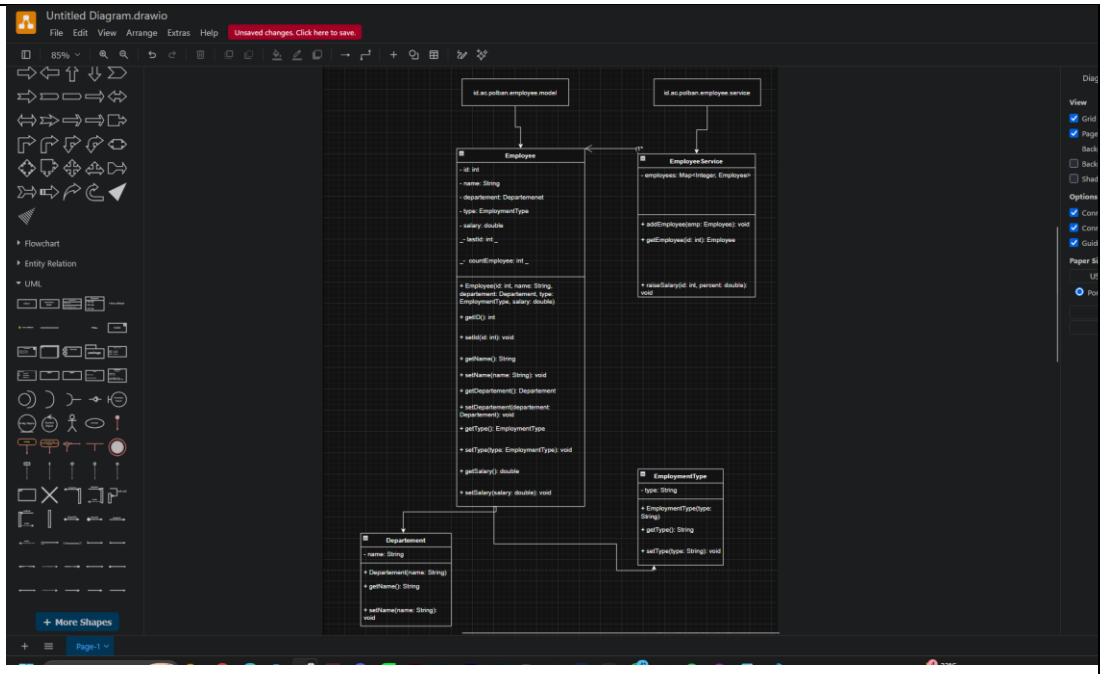
   Solusi: Menerapkan static field lastId untuk urutan ID dan countEmployees untuk total objek, serta static method getCountEmployees() untuk mengakses data tersebut secara global dari kelas Employee.

2. Permasalahan: Struktur file yang berantakan dalam satu folder menyulitkan pemisahan antara data dan logika bisnis.

   Solusi: Membagi kelas ke dalam dua paket standar: id.ac.polban.employee.model untuk entitas data dan id.ac.polban.employee.service untuk aturan bisnis.

3. Permasalahan: Dibutuhkan visualisasi teknis untuk memetakan atribut, metode, dan hubungan antar kelas agar sistem mudah dipahami pengembang lain.

   Solusi: Membuat diagram UML yang merinci struktur kelas Employee, Department, EmploymentType, dan EmployeeService beserta tipe data dan visibilitasnya (+/-).



4. Permasalahan: Bagaimana cara menyambungkan antar kelas agar dapat bertukar data sesuai kebutuhan fungsional?

   Solusi: Menerapkan Dependency melalui parameter pada method addEmployee dan Aggregation dengan menggunakan koleksi Map<Integer, Employee> sebagai penampung data permanen di EmployeeService.

5. Permasalahan: Perlunya pemahaman teoritis mengapa jenis relasi tertentu dipilih untuk kasus tertentu.

   Solusi:Aggregation: Hubungan kepemilikan di mana objek di dalamnya tetap bisa berdiri sendiri.
   Dependency: Hubungan ketergantungan sementara di mana satu kelas membutuhkan kelas lain sebagai pendukung operasional (parameter).

6. Permasalahan: Bagaimana cara mengemas seluruh *source code* menjadi satu file tunggal yang siap didistribusikan?

   Solusi: Menggunakan fitur *Export to JAR* pada VS Code dengan menentukan Main.java sebagai titik awal aplikasi.

7. Permasalahan: Memastikan file JAR yang dihasilkan benar-benar berfungsi dan dapat diakses sebagai library di luar proyek aslinya.

   Solusi: Membuat proyek baru, memasukkan JAR ke Referenced Libraries, dan menjalankan kelas penguji App.java. Hasil terminal menunjukkan status: "File JAR bekerja 100% sempurna!".

```
=== MENGUJI FILE JAR ===
Berhasil menambahkan: Budi
Total Karyawan saat ini: 1
STATUS: File JAR bekerja 100% sempurna!
```

Nama Teman dan Hal yang Dibantu (Opsional)
Hafiz(menjelaskan tentang UML dan apa saja yang harus di isi di dalam Main.java)